

INFORME TÉCNICO

Clasificador Vehicular mediante Redes Neuronales Convolucionales (CNN)

Detección de Flotas Logísticas mediante Clasificación Vehicular

Desarrollador: Tec. Mario Ezequiel Arce Especialista en Inteligencia Artificial y Ciencia de Datos

Archivo del modelo: ev3_cnn_vehicular_final.keras

Fecha: Julio 2026

Índice

- ▶ 1. Resumen
- ▶ 2. Introducción y contexto del problema
- ▶ 3. Marco teórico y justificación de la arquitectura
- ▶ 4. Metodología
- ▶ 5. Resultados
- ▶ 6. Discusión
- ▶ 7. Conclusiones
- ▶ 8. Recomendaciones y trabajo futuro
- ▶ 9. Anexo técnico

1. Resumen ejecutivo

Este informe documenta el desarrollo, entrenamiento y evaluación de un clasificador binario de vehículos basado en una Red Neuronal Convolutiva (CNN), orientado a estimar la composición de una flota logística en dos categorías: vehículos Livianos (automóviles) y vehículos Pesados (camiones).

El modelo fue entrenado sobre un subconjunto filtrado del dataset CIFAR-10 (clases automobile y truck), utilizando técnicas modernas de regularización: aumento de datos integrado en la red (Data Augmentation), normalización por lotes (Batch Normalization), Dropout progresivo, Global Average Pooling y callbacks adaptativos de entrenamiento (Early Stopping, Reduce LR on Plateau y Model Checkpoint).

Los resultados obtenidos sobre el conjunto de prueba (2.000 imágenes, no vistas durante el entrenamiento) muestran un desempeño sólido y estable:

Métrica	Valor
Accuracy (exactitud)	94.40 %
Precisión (promedio ponderado)	0.94
Recall (promedio ponderado)	0.94
F1-score (promedio ponderado)	0.94
AUC-ROC	0.9875

Estos resultados validan la hipótesis central del proyecto: una arquitectura convolutiva, al aprender filtros espaciales jerárquicos en lugar de tratar la imagen como un vector plano, logra capturar las diferencias morfológicas entre siluetas de vehículos livianos y pesados de forma sustancialmente más efectiva que un perceptrón multicapa clásico.

2. Introducción y contexto del problema

2.1 Motivación

En logística y gestión de flotas, contar con una estimación automática y confiable de la composición vehicular (proporción de vehículos livianos vs. pesados) permite optimizar la planificación de rutas, el mantenimiento preventivo, el consumo de combustible y la asignación de recursos. El presente proyecto simula este escenario aplicando visión por computadora sobre imágenes de vehículos, con el objetivo de construir un sistema capaz de clasificar automáticamente cada unidad observada.

2.2 Objetivo general

Diseñar, entrenar y evaluar una Red Neuronal Convolutiva capaz de clasificar imágenes de vehículos en dos categorías —Liviano y Pesado— e integrar los resultados del modelo en un indicador (KPI) de composición de flota.

2.3 Objetivos específicos

- Filtrar y re-etiquetar el dataset público CIFAR-10 para obtener un problema de clasificación binaria representativo (automobile → Liviano, truck → Pesado).
- Aplicar operadores clásicos de detección de bordes (Sobel, Laplaciano, Canny) como análisis exploratorio previo, para justificar la necesidad de una arquitectura con capacidad de aprender invarianzas espaciales.

- Construir una arquitectura CNN moderna con bloques convolucionales, normalización por lotes, regularización por Dropout y Global Average Pooling.
- Entrenar el modelo con callbacks adaptativos (Early Stopping, Reduce LR on Plateau, Model Checkpoint) y un esquema de validación explícito (80/20).
- Evaluar el modelo con métricas estándar de clasificación (accuracy, precisión, recall, F1-score, matriz de confusión, curva ROC y AUC).
- Traducir el desempeño del modelo a un KPI de negocio: proporción estimada de vehículos pesados en la flota analizada.

3. Marco teórico y justificación de la arquitectura

3.1 De MLP a CNN

Un modelo baseline (MLP) trataba cada imagen como un vector unidimensional de píxeles, perdiendo toda la información de adyacencia espacial entre ellos. Este enfoque impone una fuerte limitación: dos imágenes con el mismo objeto desplazado unos pocos píxeles son representadas por vectores completamente distintos, obligando a la red a re-aprender el mismo patrón en cada posición posible.

Las Redes Neuronales Convolucionales (CNN) resuelven esta limitación mediante filtros convolucionales compartidos que se deslizan sobre la imagen, preservando la estructura espacial y aprendiendo jerarquías de características: bordes y texturas simples en las primeras capas, formas y contornos en capas intermedias, y estructuras de alto nivel (siluetas, proporciones) en las capas más profundas.

3.2 Análisis exploratorio con operadores clásicos de bordes

Antes de construir la CNN, se aplicaron operadores clásicos de procesamiento de imágenes —**Sobel** (primera derivada), **Laplaciano del Gaussiano** (LoG) y el algoritmo de **Canny**— sobre muestras representativas de ambas clases. Este análisis exploratorio permitió observar de forma directa las diferencias morfológicas entre siluetas: los vehículos livianos presentan bordes curvos y perfil bajo, mientras que los vehículos pesados muestran bordes angulares, cabina elevada y caja de carga de geometría rectangular.

Estas diferencias, visibles mediante operadores fijos y no entrenables, constituyen la justificación empírica para emplear una CNN: los filtros convolucionales entrenables de la red son capaces de aprender automáticamente detectores equivalentes —y más ricos— a los operadores de Sobel y Canny aplicados manualmente, ajustándose específicamente al problema durante el entrenamiento.

3.3 Componentes clave de la arquitectura

- **Data Augmentation** integrada en el modelo (**RandomFlip**, **RandomRotation**, **RandomZoom**): se ejecuta en GPU como parte del grafo de la red y solo está activa durante el entrenamiento, aumentando la variabilidad efectiva del dataset sin costo de preprocesamiento externo.
- Bloques Conv2D + BatchNormalization: la normalización por lotes estabiliza y acelera el entrenamiento al normalizar las activaciones intermedias, permitiendo tasas de aprendizaje más altas.
- MaxPooling2D: reduce progresivamente la resolución espacial, disminuyendo el costo computacional y aportando invariancia a pequeñas traslaciones.
- Dropout progresivo (0.25 → 0.25 → 0.40 → 0.50): regularización que se intensifica en las capas más profundas, donde el riesgo de sobreajuste es mayor.

- GlobalAveragePooling2D: reemplaza el clásico Flatten + Dense de gran tamaño, reduciendo drásticamente el número de parámetros y el riesgo de overfitting, al promediar cada mapa de características a un único escalar.
- Salida sigmoide + Binary Crossentropy: apropiada para el problema de clasificación binaria (Liviano/Pesado), a diferencia de la salida Softmax de 2 neuronas utilizada en el MLP baseline.

4. Metodología

4.1 Dataset y preprocesamiento

Se utilizó el dataset público **CIFAR-10** (60.000 imágenes RGB de 32×32 píxeles en 10 clases), del cual se filtraron únicamente las clases automobile (etiqueta original 1) y truck (etiqueta original 9), re-etiquetadas como Liviano (0) y Pesado (1) respectivamente. Tras el filtrado se obtuvieron aproximadamente 10.000 imágenes de entrenamiento y 2.000 de prueba, balanceadas entre ambas clases.

Las imágenes fueron normalizadas mediante escalado Min-Max, transformando el rango original [0, 255] a [0.0, 1.0], lo cual favorece la estabilidad numérica y la velocidad de convergencia del entrenamiento.

4.2 Esquema de validación

Conjunto	N.º de imágenes	Pasos por época	Uso
Entrenamiento	8.000 (80 %)	250	Ajuste de pesos
Validación	2.000 (20 %)	63	Monitoreo de overfitting / callbacks
Prueba (test)	2.000	63	Evaluación final, no vista en entrenamiento

El conjunto de entrenamiento original (10.000 imágenes) se dividió explícitamente en 80 % entrenamiento / 20 % validación (semilla fija = 42), separado del conjunto de prueba oficial de CIFAR-10 ya filtrado, garantizando que la evaluación final se realice sobre datos completamente no vistos por el modelo durante el ajuste de pesos ni durante la selección de hiperparámetros.

4.3 Arquitectura del modelo (CNN_Vehicular_v3)

La red fue implementada como un modelo Sequential de Keras/TensorFlow, con la siguiente estructura de bloques:

Bloque	Composición	Parámetros aprox.
Entrada	Input(32, 32, 3)	—
Aumento de datos	RandomFlip, RandomRotation(0.10), RandomZoom(0.10)	0
Bloque Conv 1	2× Conv2D(32,3×3) + BN + MaxPool(2×2) + Dropout(0.25)	10,400
Bloque Conv 2	2× Conv2D(64,3×3) + BN + MaxPool(2×2) + Dropout(0.25)	55,936
Bloque Conv 3	2× Conv2D(128,3×3) + BN + MaxPool(2×2) + Dropout(0.40)	222,464
Cabeza de clasificación	GlobalAveragePooling2D → Dense(256) + BN + Dropout(0.50)	34,048

Bloque	Composición	Parámetros aprox.
Salida	Dense(1, sigmoid)	257

Indicador	Valor
Parámetros totales	323,105
Parámetros entrenables	321,697
Parámetros no entrenables (BatchNorm)	1,408
Tamaño del modelo (pesos)	≈ 1.23 MB
Nombre del modelo	CNN_Vehicular_v3

4.4 Compilación y configuración de entrenamiento

- Optimizador: Adam, tasa de aprendizaje inicial 1e-3.
- Función de pérdida: Binary Crossentropy (coherente con la salida sigmoide de una sola neurona).
- Métricas monitoreadas: Accuracy, Precision, Recall y AUC.
- Épocas máximas: 50 (el entrenamiento se ejecutó hasta completar las 50 épocas programadas).
- Tamaño de lote (batch size): 32.

Callbacks utilizados

Callback	Configuración	Función
EarlyStopping	monitor=val_loss, patience=8, restore_best_weights=True	Evita sobreajuste; recupera los mejores pesos
ReduceLROnPlateau	monitor=val_loss, factor=0.5, patience=4, min_lr=1e-6	Reduce la tasa de aprendizaje ante estancamiento
ModelCheckpoint	monitor=val_accuracy, save_best_only=True	Persiste el mejor modelo según validación

Durante el entrenamiento, ReduceLROnPlateau redujo la tasa de aprendizaje en cuatro oportunidades (épocas 20, 27, 34 y 44), llevándola progresivamente de 1e-3 a 6.25e-5, lo que permitió refinar la convergencia en las últimas etapas del entrenamiento sin producir oscilaciones bruscas en la pérdida de validación.

5. Resultados

5.1 Evolución del entrenamiento

El entrenamiento se ejecutó durante las 50 épocas programadas, sin activación anticipada de Early Stopping, lo cual indica que la pérdida de validación continuó mejorando (o manteniéndose estable) a lo largo de todo el proceso. El mejor punto según val_accuracy se registró en la época 46 (restauración de pesos desde la época 48, criterio interno de Keras), con los siguientes valores:

Métrica	Entrenamiento (última época)	Validación (mejor punto)
Loss	0.1036	0.0748
Accuracy	0.9579	0.9740
Precision	0.9558	0.9591
Recall	0.9603	0.9874
AUC	0.9932	0.9965

La evolución de las curvas de pérdida y accuracy muestra una convergencia estable, sin señales significativas de overfitting: la curva de validación se mantiene consistentemente cercana —e incluso, en varios tramos, por debajo— de la curva de entrenamiento, efecto propio del uso combinado de Data Augmentation, Dropout progresivo y Batch Normalization.

5.2 Evaluación formal sobre el conjunto de prueba

Sobre el conjunto de prueba (2.000 imágenes no vistas), el modelo alcanzó los siguientes resultados agregados:

Métrica	Valor
Loss (Binary Crossentropy)	0.1571
Accuracy	0.9440 (94.40 %)

5.3 Reporte de clasificación detallado

Clase	Precisión	Recall	F1-score	Soporte
Liviano	0.95	0.94	0.94	1.000
Pesado	0.94	0.95	0.94	1.000
Accuracy global	—	—	0.94	2.000
Macro avg	0.94	0.94	0.94	2.000
Weighted avg	0.94	0.94	0.94	2.000

El equilibrio entre precisión y recall en ambas clases (0.94–0.95 en todos los casos) indica que el modelo no favorece sistemáticamente a ninguna de las dos categorías, un resultado esperable dado que el dataset filtrado está balanceado (1.000 imágenes de cada clase en el conjunto de prueba).

5.4 Matriz de confusión y curva ROC

A partir del reporte de clasificación y el desglose de predicciones se reconstruye la matriz de confusión aproximada sobre las 2.000 imágenes de prueba:

	Predicho: Liviano	Predicho: Pesado	Total real
Real: Liviano	935 (VN)	65 (FP)	1.000
Real: Pesado	47 (FN)	953 (VP)	1.000

El área bajo la curva ROC (AUC-ROC) obtenida fue de 0.9875, un valor muy cercano al máximo teórico (1.0), lo que confirma que el modelo posee una excelente capacidad de discriminación entre ambas clases en prácticamente todo el rango de umbrales de decisión, no solo en el umbral por defecto de 0.5.

5.5 KPI logístico: composición de la flota

Como aplicación práctica del clasificador, se calculó un indicador de negocio directamente derivado de las predicciones del modelo sobre el conjunto de prueba, interpretado como una muestra representativa de una flota vehicular:

Clase	Cantidad real	Cantidad predicha	% predicho
Liviano	1.000	982	49.1 %
Pesado	1.000	1.018	50.9 %
Total	2.000	2.000	100 %

El modelo estima que un 50.9 % de la flota analizada corresponde a vehículos Pesados, frente a una proporción real de 50.0 %, lo que representa una desviación de apenas 0.90 puntos porcentuales. Este nivel de precisión agregada resulta más que suficiente para sustentar decisiones operativas de planificación logística (por ejemplo, dimensionamiento de infraestructura de mantenimiento diferenciada por tipo de vehículo).

5.6 Validación cualitativa: inferencia individual

Se implementó y verificó un pipeline de inferencia end-to-end sobre imágenes individuales, validando explícitamente el manejo correcto de la dimensión de lote (batch dimension) requerida por Keras al pasar de una imagen aislada (32, 32, 3) a un lote de tamaño 1 (1, 32, 32, 3). A modo de ejemplo, sobre una imagen de prueba de clase real Pesado, el modelo predijo correctamente la clase Pesado con una confianza de 99.9 %, frente a 0.1 % asignado a la clase Liviano.

6. Discusión

6.1 Fortalezas del enfoque

- La combinación de Batch Normalization, Dropout progresivo y Data Augmentation integrada logró un entrenamiento estable a lo largo de 50 épocas, sin señales de divergencia ni overfitting severo.
- El uso de Global Average Pooling en lugar de Flatten + Dense mantuvo el modelo compacto (323 mil parámetros), reduciendo el riesgo de sobreajuste sin sacrificar capacidad predictiva.
- El **AUC-ROC de 0.9875** indica que el modelo es robusto ante distintos umbrales de decisión, lo que otorga flexibilidad operativa: el umbral de clasificación podría ajustarse según el costo relativo de falsos positivos y falsos negativos en un despliegue real.
- El KPI logístico derivado (desviación de 0.90 puntos porcentuales respecto a la proporción real) demuestra que, más allá de las métricas de clasificación estándar, el modelo es útil para estimar indicadores agregados de negocio con alta fidelidad.

6.2 Limitaciones

- El dataset utilizado (CIFAR-10, 32×32 píxeles) tiene una resolución muy inferior a la de imágenes reales capturadas por cámaras de tránsito o peajes, por lo que el desempeño reportado debe entenderse como una prueba de concepto y no como una validación en condiciones de producción.

- Solo se contemplan dos categorías (Liviano/Pesado); una flota real incluye subcategorías adicionales (motos, utilitarios, buses, semirremolques) no representadas en el problema binario actual.
- Las imágenes de CIFAR-10 presentan condiciones de iluminación, ángulo y fondo controladas; un despliegue real requeriría validar el modelo frente a condiciones climáticas adversas, oclusiones parciales y ángulos de cámara variables.

7. Conclusiones

El desarrollo confirma que una arquitectura de Red Neuronal Convolucional, diseñada con buenas prácticas de regularización moderna (Batch Normalization, Dropout progresivo, Data Augmentation en GPU y Global Average Pooling), supera de forma sustancial y consistente al modelo MLP baseline para la tarea de clasificación vehicular sobre imágenes.

El modelo final (CNN_Vehicular_v3) alcanzó un 94.40 % de accuracy y un AUC-ROC de 0.9875 sobre un conjunto de prueba independiente, con métricas de precisión y recall equilibradas entre ambas clases (0.94–0.95), lo que indica un clasificador robusto y sin sesgos relevantes hacia ninguna categoría.

Más allá de las métricas técnicas, el proyecto demuestra la aplicabilidad directa de este tipo de modelos a un caso de uso de negocio concreto: la estimación automática de la composición de una flota logística, con una desviación de menos de un punto porcentual respecto al valor real, lo cual valida el pipeline completo, desde el preprocesamiento de datos hasta la traducción de las predicciones en un KPI accionable.

El modelo entrenado fue serializado y entregado como artefacto reutilizable (ev3_cnn_vehicular_final.keras), quedando disponible para su integración en procesos de inferencia batch o en tiempo real.

8. Recomendaciones y trabajo futuro

- Ampliar el dataset con imágenes de mayor resolución y condiciones reales de captura (cámaras de tránsito, distintos ángulos y climas) para validar la capacidad de generalización del modelo fuera del entorno controlado de CIFAR-10.
- Extender el problema a clasificación multiclase, incorporando categorías adicionales relevantes para gestión de flotas (motocicletas, utilitarios, buses, acoplados).
- Explorar técnicas de Transfer Learning (por ejemplo, backbones preentrenados como MobileNetV2 o EfficientNet) para evaluar si aportan mejoras adicionales de exactitud o eficiencia computacional frente a la arquitectura entrenada desde cero en este proyecto.
- Incorporar técnicas de interpretabilidad (Grad-CAM) para visualizar qué regiones de la imagen utiliza la CNN al momento de decidir entre Liviano y Pesado, reforzando la confianza y la explicabilidad del sistema ante usuarios no técnicos.
- Evaluar el modelo en un entorno de inferencia en tiempo real (streaming de video), midiendo latencia y throughput, dado el tamaño reducido del modelo (≈ 1.2 MB de pesos entrenables).

9. Anexo técnico

9.1 Entorno de ejecución

Componente	Detalle
Framework	TensorFlow 2.20.0 / Keras 3

Componente	Detalle
Aceleración	GPU (physical_device:GPU:0)
Dataset	CIFAR-10 (tensorflow_datasets), clases 1 (automobile) y 9 (truck)
Bibliotecas de evaluación	scikit-learn (classification_report, confusion_matrix, roc_curve, roc_auc_score)
Bibliotecas de análisis exploratorio	OpenCV (cv2): Sobel, Laplaciano, Canny

9.2 Artefactos generados

- `ev3_cnn_vehicular_final.keras` — modelo completo (arquitectura + pesos finales + estado del optimizador).
- `mejor_modelo_cnn_vehicular.keras` — checkpoint del mejor modelo según `val_accuracy` durante el entrenamiento.
- `ev3_veh_classifier_cnn.ipynb` — notebook completo con las 19 secciones documentadas: importaciones, carga y filtrado del dataset, normalización, análisis exploratorio, arquitectura, compilación, callbacks, entrenamiento, curvas de aprendizaje, evaluación, métricas avanzadas, KPI logístico, visualización de predicciones, inferencia individual y comparación con el baseline.

9.3 Ficha técnica del modelo

Campo	Valor
Nombre interno	CNN_Vehicular_v3
Forma de entrada	(32, 32, 3)
Tipo de salida	Sigmoide (probabilidad de clase Pesado)
Umbral de decisión	0.5
Parámetros entrenables	321,697
Parámetros no entrenables	1,408
Tamaño en disco (modelo completo)	≈ 3.7 MB (incluye estado del optimizador Adam)